

Projeto Prático de Ciência de Dados

Professor José Antonio de Paiva Júnior

Matheus Martins – 22252565

Thales Martins – 22252016

Brasília

Outubro de 2025

Análise de Performance de Jogadores de Futebol

Resumo

- O presente projeto tem como objetivo analisar o desempenho de jogadores de futebol por meio da coleta, tratamento e análise de dados, aplicando técnicas de ciência de dados e aprendizado de máquina. Inicialmente, os dados foram coletados de múltiplas fontes, integrados e submetidos a processos de limpeza, padronização e transformação, garantindo a consistência e qualidade do conjunto de informações.
- Foram calculadas estatísticas descritivas e métricas básicas, como média, mediana, moda e desvio padrão, permitindo compreender o comportamento geral das variáveis de desempenho. Além disso, foram definidos e construídos indicadores de desempenho (KPIs), como gols por partida, assistências por partida e um KPI geral (“overall KPI”), que sintetiza o desempenho ofensivo, defensivo e de passes dos jogadores.
- O projeto também envolveu a visualização dos dados, por meio de gráficos e dashboards, possibilitando a identificação de padrões, tendências e insights relevantes para avaliação do desempenho dos atletas. Técnicas de feature engineering foram aplicadas para criação de novas variáveis, melhorando a qualidade da modelagem preditiva.
- Foram implementados modelos de regressão linear e logística, árvores de decisão e KNN, com o objetivo de prever indicadores de desempenho e classificar jogadores em diferentes categorias. Algoritmos de clusterização (K-Means e DBSCAN) e regras de associação (Apriori) também foram utilizados para identificar agrupamentos naturais e relações significativas entre indicadores.
- Por fim, métricas de avaliação de modelos, como precisão, recall, F1-score, matriz de confusão e validação cruzada, foram calculadas para medir a performance dos modelos e prevenir problemas de overfitting. O projeto fornece uma base sólida para análises de desempenho de jogadores e pode ser expandido para suporte a decisões em clubes, scouts e estratégias de treinamento.

1. Coleta de Dados

- Fonte dos dados - Kaggle

<https://www.kaggle.com/datasets/cihan063/soccer-players>

<https://www.kaggle.com/datasets/hubertsidorowicz/football-players-stats-2024-2025>

- Os dados foram coletados do Kaggle, foram usados dois datasets públicos sobre os status de jogadores de futebol.
-

2. Limpeza e Pré-processamento de Dados

- **2.1 Limpeza**

- Tratamento de valores ausentes
- Remoção de duplicatas

- **2.2 Integração**

- Merge das bases selecionando apenas jogadores comuns nos dois datasets
- Padronização de colunas

- **2.3 Transformação**

- Conversão de tipos
 - Normalização e padronização
-

3. Estatísticas Descritivas e Métricas Básicas

- Comparações entre atributos ofensivos, defensivos e físicos dos jogadores
- Média, mediana, moda, desvio padrão, valores mínimo e máximo das colunas selecionadas

```
def descriptive_statistics(df):  🧑 Thales Martins
    numeric_cols = df.select_dtypes(include='number').columns

    stats = pd.DataFrame(index=numeric_cols)
    stats['mean'] = df[numeric_cols].mean()
    stats['median'] = df[numeric_cols].median()
    stats['mode'] = df[numeric_cols].mode().iloc[0]
    stats['std'] = df[numeric_cols].std()
    stats['min'] = df[numeric_cols].min()
    stats['max'] = df[numeric_cols].max()
    return stats
```

4. Definição e Construção de Indicadores (KPIs)

Dividido em cinco categorias básicas totalizando 20 KPIs

- KPIs ofensivos: *goals_per_match*, *assists_per_match*, *contrib_off*
- KPIs defensivos: *tackles_per_match*, *interceptions_per_match*, *clean_tackles*
- KPIs de passe: *passes_per_match*, *key_passes_per_match*, *pass_accuracy*
- KPIs físicos: *speed*, *stamina_value*, *strength_value*
- KPI geral (*overall_kpi*): fórmula e ponderação

```

def calculate_offensive_kpis(df):...

def calculate_defensive_kpis(df):...

def calculate_passing_kpis(df):...

def calculate_physical_kpis(df):...

def calculate_goalkeeper_kpis(df):...

def calculate_overall_kpi(df): 1 usage 🧑 Thales Martins
    df['overall_kpi'] = (
        df.get('contrib_off_per_match', 0) * 0.3 +
        df.get('tackles_per_match', 0) * 0.2 +
        df.get('pass_accuracy', 0) * 0.2 +
        df.get('speed', 0) * 0.1 +
        df.get('physical_intensity', 0) * 0.1 +
        df.get('duels_won_rate', 0) * 0.1
    )
    return df

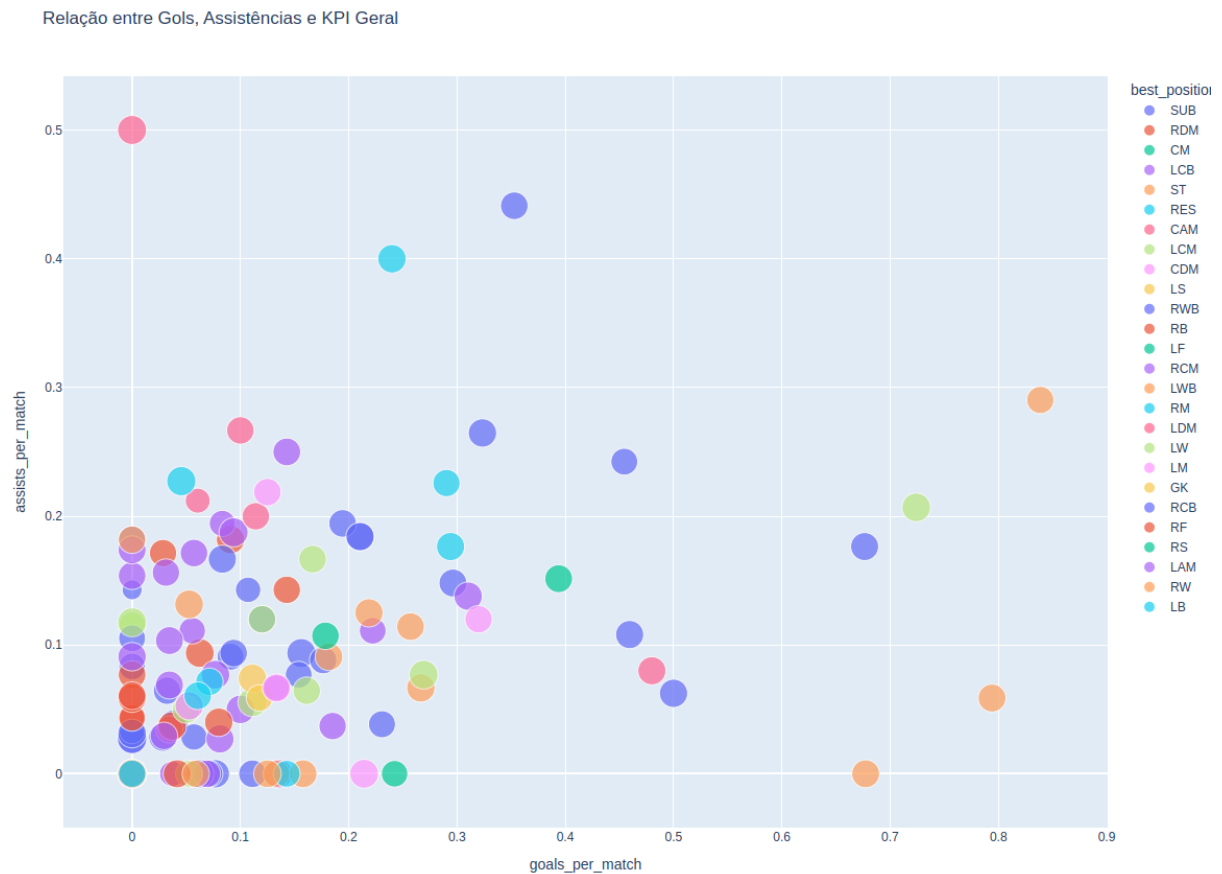
def calculate_all_kpis(df):...

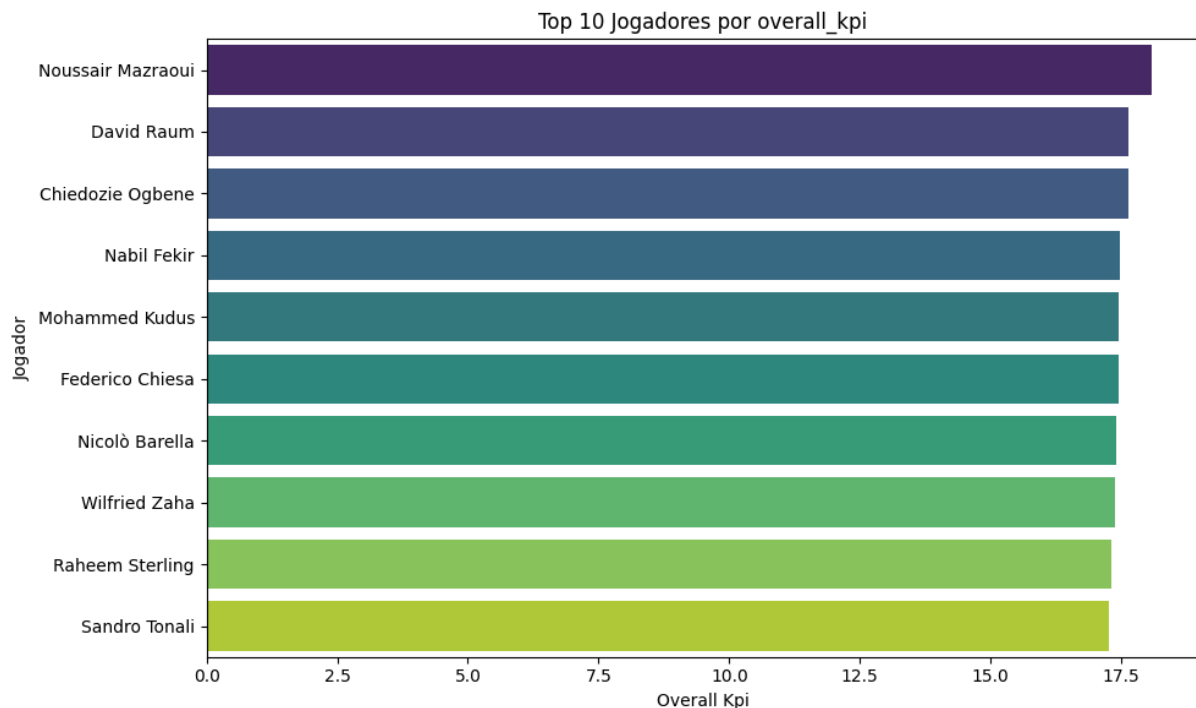
def top_players_by_kpi(df, kpi='overall_kpi', top_n=10):...

```

5. Visualização de Dados e Dashboards

- Histogramas, scatter plots, boxplots
- Dashboard: visão geral de KPIs e desempenho
- Insights gerados por storytelling visual





6. Transformação de Dados e Feature Engineering

Feature Engineering refere-se à criação de novas variáveis (features) a partir dos dados existentes, de forma a fornecer mais informações relevantes para análise ou modelagem preditiva.

```
feature_cols = [  
    'goals_per_match', 'assists_per_match', 'passes_per_match',  
    'tackles_per_match', 'pass_accuracy', 'speed', 'overall_kpi'  
]
```

Transformação binária:

Certos KPIs foram convertidos em variáveis binárias para simplificar a identificação de alto desempenho. Valor 1 para jogadores cujo certo KPI está acima do valor comparativo, e 0 caso o contrário. Exemplo:

```
df_bin = pd.DataFrame({
    'high_goals': df['goals_per_match'] > 0.3,
    'high_assists': df['assists_per_match'] > 0.2,
    'high_pass_accuracy': df['pass_accuracy'] > 0.8,
    'high_speed': df['speed'] > 70,
    'high_overall': df['overall_kpi'] > df['overall_kpi'].median()
})
```

7. Modelagem Preditiva

7.1 Regressão Linear, Logística, Árvores de Decisão e KNN

```
def run_modeling():

    df = pd.read_csv("processed_data/players_with_kpis.csv")

    regression_features = ["crossing", "finishing", "short_passing",
                           "overall_kpi"]

    regression_target = "goals_per_match"

    df = df.dropna(subset=regression_features + [regression_target])

    X = df[regression_features]
    y = df[regression_target]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                         random_state=42)

    linear_model = LinearRegression()

    linear_model.fit(X_train, y_train)
```



```

y_pred = linear_model.predict(X_test)

print("[Regressão Linear]")

print(f"R²: {r2_score(y_test, y_pred):.4f}")

print(f"MSE: {mean_squared_error(y_test, y_pred):.4f}\n")

df["high_scorer"] = (df["goals_per_match"] > 0.3).astype(int)

logistic_features = regression_features

logistic_target = "high_scorer"

X = df[logistic_features]

y = df[logistic_target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

logistic_model = LogisticRegression(max_iter=1000)

logistic_model.fit(X_train, y_train)

y_pred = logistic_model.predict(X_test)

print("[Regressão Logística]")

print(f"Acurácia: {accuracy_score(y_test, y_pred):.4f}")

print(classification_report(y_test, y_pred, zero_division=0))

tree_model = DecisionTreeClassifier(max_depth=4, random_state=42)

tree_model.fit(X_train, y_train)

y_pred_tree = tree_model.predict(X_test)

```

```
print("[Árvore de Decisão]")

print(f"Acurácia: {accuracy_score(y_test, y_pred_tree):.4f}")

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

y_pred_knn = knn_model.predict(X_test)

print("[KNN]")

print(f"Acurácia: {accuracy_score(y_test, y_pred_knn):.4f}")

print("\nModelagem preditiva concluída!")
```

Resultados:

```
[Regressão Linear]
```

```
R²: 0.2136
```

```
MSE: 0.0233
```

```
[Regressão Logística]
```

```
Acurácia: 0.9062
```

	precision	recall	f1-score	support
0	0.91	1.00	0.95	29
1	0.00	0.00	0.00	3
accuracy			0.91	32
macro avg	0.45	0.50	0.48	32
weighted avg	0.82	0.91	0.86	32

```
[Árvore de Decisão]
```

```
Acurácia: 0.8750
```

```
[KNN]
```

```
Acurácia: 0.9062
```

```
Modelagem preditiva concluída!
```

8. Algoritmos de Machine Learning

- Clusterização é uma técnica de aprendizado não supervisionado.
- O objetivo é agrupar objetos semelhantes em clusters, sem precisar de rótulos pré-definidos.

Exemplos de algoritmos:

1. K-Means

- Divide os dados em K grupos.
- Cada ponto pertence ao cluster mais próximo de um centroide.

- No código:

```
def run_ml_analysis(): 2 usages Thales Martins
    df = pd.read_csv('processed_data/players_with_kpis.csv')

    kpi_cols = ['goals_per_match', 'assists_per_match', 'passes_per_match', 'tackles_per_match', 'pass_accuracy', 'speed', 'overall_kpi']

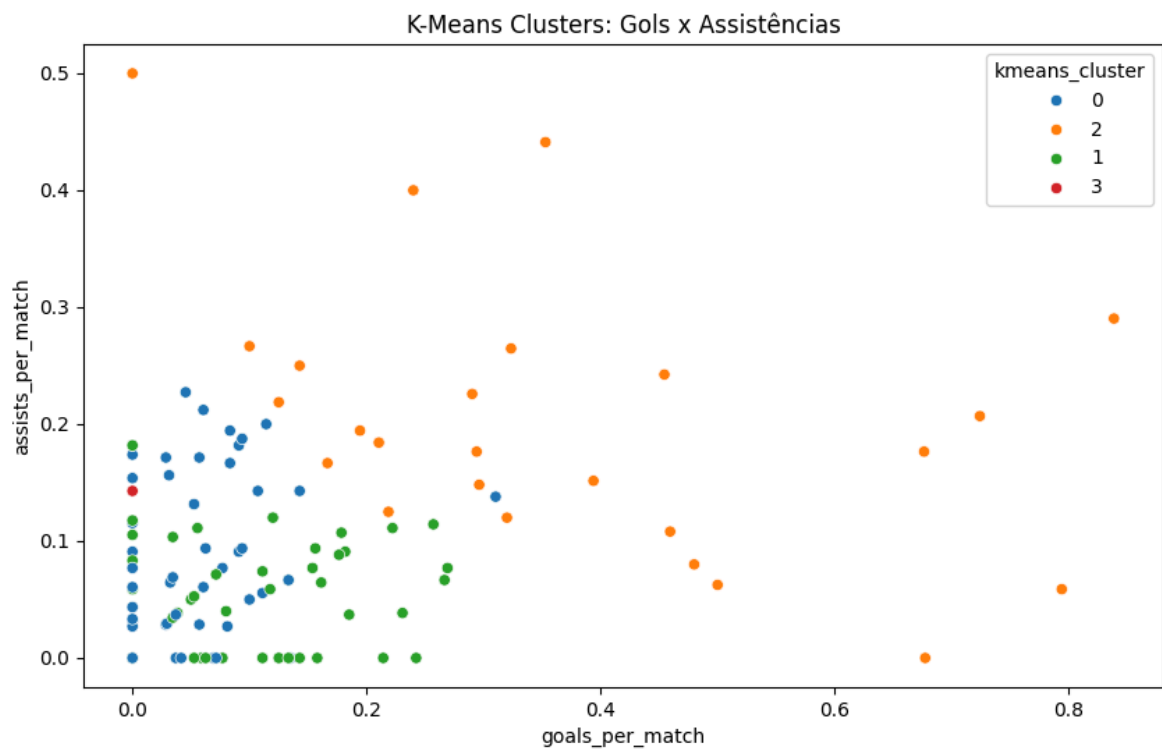
    X = df[kpi_cols].fillna(0)

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    kmeans = KMeans(n_clusters=4, random_state=42)
    df['kmeans_cluster'] = kmeans.fit_predict(X_scaled).astype(str)
    print("K-Means cluster distribution:")
    print(df['kmeans_cluster'].value_counts())

    plt.figure(figsize=(10, 6))
    sns.scatterplot(x=df['goals_per_match'], y=df['assists_per_match'],
                    hue=df['kmeans_cluster'], palette='tab10')
    plt.title('K-Means Clusters: Gols x Assistências')
    plt.show()
```

- Aqui, os jogadores são agrupados pelos KPIs
- Depois usamos um scatter plot para visualizar os clusters



2. DBSCAN (Density-Based Spatial Clustering)

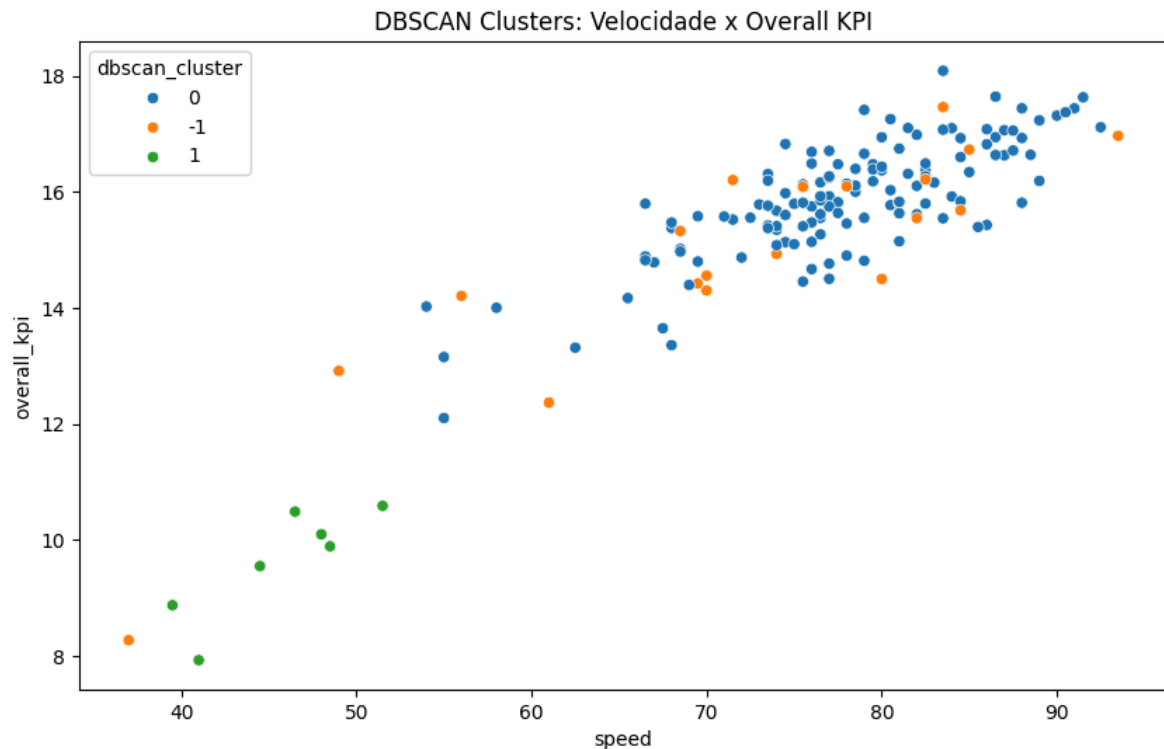
- Agrupa pontos que estão densamente próximos
- Identifica outliers como pontos que não pertencem a nenhum cluster
- Funciona bem quando os clusters têm formas irregulares

- No código:

```
dbscan = DBSCAN(eps=1.5, min_samples=5)
df['dbscan_cluster'] = dbscan.fit_predict(X_scaled).astype(str)
print("DBSCAN cluster distribution (-1 = outlier):")
print(df['dbscan_cluster'].value_counts())

plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['speed'], y=df['overall_kpi'],
                hue=df['dbscan_cluster'], palette='tab10')
plt.title('DBSCAN Clusters: Velocidade x Overall KPI')
plt.show()
```

- Os jogadores que não se encaixam em nenhum grupo ficam com o `dbscan_cluster = -1`



3. Regras de Associação (Apriori)

Conceito:

- Apriori é usado para encontrar padrões frequentes em conjuntos de itens.
- Ajuda a descobrir relações do tipo “se A acontece, B também acontece”.

Exemplo:

- Se um jogador tem um alto aproveitamento de passes e um overall elevado ele provavelmente também será um jogador veloz.

No código:

```

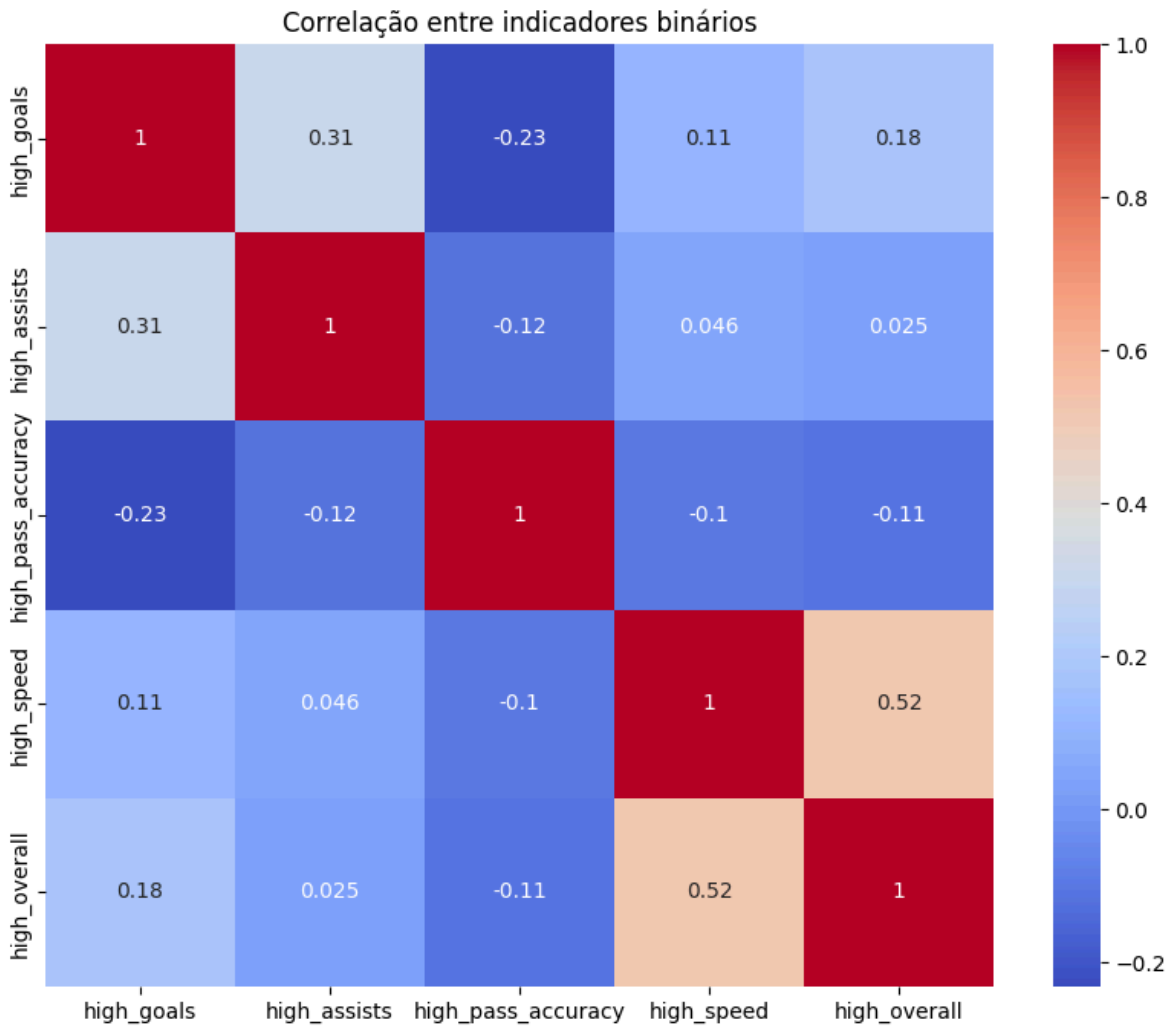
df_bin = pd.DataFrame({
    'high_goals': df['goals_per_match'] > 0.3,
    'high_assists': df['assists_per_match'] > 0.2,
    'high_pass_accuracy': df['pass_accuracy'] > 0.8,
    'high_speed': df['speed'] > 70,
    'high_overall': df['overall_kpi'] > df['overall_kpi'].median()
})

df_bin = df_bin.astype(bool)
frequent_itemsets = apriori(df_bin, min_support=0.1, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
print("\nApriori Association Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])

plt.figure(figsize=(10, 8))
sns.heatmap(df_bin.corr(), annot=True, cmap='coolwarm')
plt.title('Correlação entre indicadores binários')
plt.show()

```

Resultados:



Métricas importantes:

- **Confidence:** probabilidade de consequente dado o antecedente.
- **Lift:** indica se a regra é mais forte que aleatória (maior que 1 = boa associação).

9. Métricas de Avaliação de Modelos

```
precision, recall, f1 = calculate_classification_metrics(y_test, y_pred)
cm, report = generate_confusion_matrix(y_test, y_pred)
cv_accuracy = run_cross_validation(model, X, y, cv_folds=5)
```

9.a Precisão, Recall, F1-Score

- **Precisão (Precision):** de todas as previsões positivas, quantas estavam corretas.
- **Recall (Sensibilidade):** de todos os positivos reais, quantos o modelo conseguiu identificar.
- **F1-Score:** média harmônica entre precisão e recall

```
✓ Métricas de Classificação:  
Precision: 0.9231 | Recall: 0.7500 | F1-Score: 0.8276
```

9.b Matriz de Confusão

- Tabela que mostra:
 - **True Positives (TP):** acertos positivos
 - **False Positives (FP):** classificados como positivos mas não eram
 - **True Negatives (TN):** acertos negativos
 - **False Negatives (FN):** classificados como negativos mas eram positivos

```
✓ Matriz de Confusão:  
[[15  1]  
 [ 4 12]]
```

9.c Validação Cruzada e Overfitting

- **Validação cruzada (cross-validation):**
 - Divide os dados em k partes, treina em k-1 e testa na parte restante, repetindo para todas as divisões.
 - Objetivo: avaliar a performance média do modelo e evitar que ele aprenda ruído específico do conjunto de treino.
- **Overfitting:**

- Quando o modelo aprende detalhes e ruído do treino, não generaliza bem para novos dados.
- **Sinais:** alta acurácia no treino e baixa no teste.

```
✓ Validação Cruzada (Acurácia média): 0.7637
```

Repositório do projeto:

- <https://github.com/tithalss/DataScienceProject>